

# Converge

A complete, evidence-led method for carrying an idea through human agreement, bounded machine execution, and independently checked settlement.

*One descent. Nine pass contracts. One conductor. One barrier.*

Converge turns ambiguous intent into durable artifacts, makes the definition of done travel with each task, and constrains an agent to the smallest authority that task earns. Every transition ends in evidence a human or machine can inspect.

## Agreement descends

Business intent, observable behavior, architecture, seams, and atomic tasks—in that order.

## Authority narrows

Each signed task earns only the evidence, tools, paths, time, and effects it requires.

## Evidence accumulates

Gate tokens, hashes, evals, holdouts, commits, and receipts preserve the real outcome.

# The contract of this handbook

*Current-state operating reference—not a roadmap presented as reality.*

Converge has several kinds of truth, and they are intentionally not interchangeable. This handbook reads the live package in order: **executable source and tests** define semantic boundaries; `cvg help` and `agent-context` establish command existence; each skill's `SKILL.md` defines its operating contract; release notes record demonstrated behavior and known gaps. Earlier PDFs provide visual lineage only.

Version rule. Release 0.1.0 names the whole package: CLI, skills, manifests, and Task-Spec ship together. Internal contracts remain independently versioned—Task-Spec `format_version: 3`, the agent contract, and `hmac-sha256-v2`.

A green gate never proves more than its stated contract. A tracker is a projection, not the backlog. The conductor sequences work but cannot declare it correct. A model's confident prose is not a receipt. A future Manager is not implied by the single-task Loop that exists today.

| CLAIM TYPE         | REQUIRED EVIDENCE                                           | INSUFFICIENT SUBSTITUTE       |
|--------------------|-------------------------------------------------------------|-------------------------------|
| Method contract    | Skill instructions, source, and tests                       | An earlier handbook alone     |
| Next pass in route | <code>cvg next</code> + workspace + selected lane           | Chat memory or guessed pass   |
| Command exists     | Current <code>cvg help</code> or <code>agent-context</code> | A row in <code>PLAN.md</code> |
| Unattended task    | Tier-1, Pass 7 profile, host evidence                       | Valid-looking YAML alone      |
| Lifecycle close    | <code>accepted: true</code> + matching result: pass receipt | Settlement token alone        |

When sources disagree, stop at the narrowest claim that can be verified. Correct the owning source before expanding the claim. Generated indexes and tracker projections may be useful—none may silently outrank the canonical artifact.

# Contents — the full descent

*Read linearly, or jump to the altitude you own.*

| PART                  | PAGES | QUESTION ANSWERED                                              |
|-----------------------|-------|----------------------------------------------------------------|
| I · Foundations       | 04–10 | Thesis, trust layers, nine passes, routes, conductor, altitude |
| II · Design passes    | 11–16 | Deep Pass 0–4: Capture through Consensus and the barrier       |
| III · Build passes    | 17–24 | Deep Pass 5–8: Tasking, Register, Bind, Loop, Tier-2           |
| IV · Operating system | 25–28 | Trust model, CLI, machine tokens, workspace roots              |
| V · Practice          | 29–33 | Walkthroughs, demonstrated truth, vocabulary, close            |

## If you own the outcome

Read foundations, then every Design pass and the barrier.  
Decisions: no-go, acceptance metrics, irreversible architecture, residual risk, owner sign-off.

## If you operate agents

Conductor, Tasking, Bind, Loop, Tier-2. Watch signatures, capability closure, stop states, and holdout independence.

## If you integrate tooling

CLI envelope, workspace resolver, adapter boundaries, stable tokens before automating.

## If you audit evidence

Artifact lineage, hashes, evals, Tier-2 verdicts, receipts, local commit vs pushed PR.

Operating shortcut. Product meaning lives in the BRD or tech spec; irreversible choices in ADRs; seams in plans; delegated behavior in Task-Specs; runtime rights in the profile; outcomes in Loop logs and receipts. Edit only the owner—renew every downstream proof.

# Reduce ambiguity before spending autonomy

*A controlled descent from owner intent to executable proof.*

Software fails when decisions change altitude without becoming explicit. A business outcome becomes an implementation preference; a plan becomes a task before seams are understood; an agent is handed a repository and told to “make it work.” Converge inserts durable contracts at each altitude so uncertainty is resolved where it is cheapest.

**01 Intent becomes falsifiable**

Outcomes, scope, constraints, and acceptance metrics before architecture.

**02 Structure becomes durable**

Hard-to-reverse decisions grounded against the live system.

**03 Work becomes decomposable**

Natural seams become swimlanes; lanes become independently provable legs.

**04 Plans are attacked before automation**

A different model family tries to refute; owner resolves every objection.

**05 Execution becomes bounded**

Each leaf carries evals, write scope, budgets, and stop conditions.

**06 Completion becomes evidence**

Green evals, path checks, receipts, optional Tier-2 replace confidence.

## Human authority

Owner decides worth, trade-offs, residual risk, and signs Pass 4. A gate exposes missing evidence; it cannot impersonate that decision.

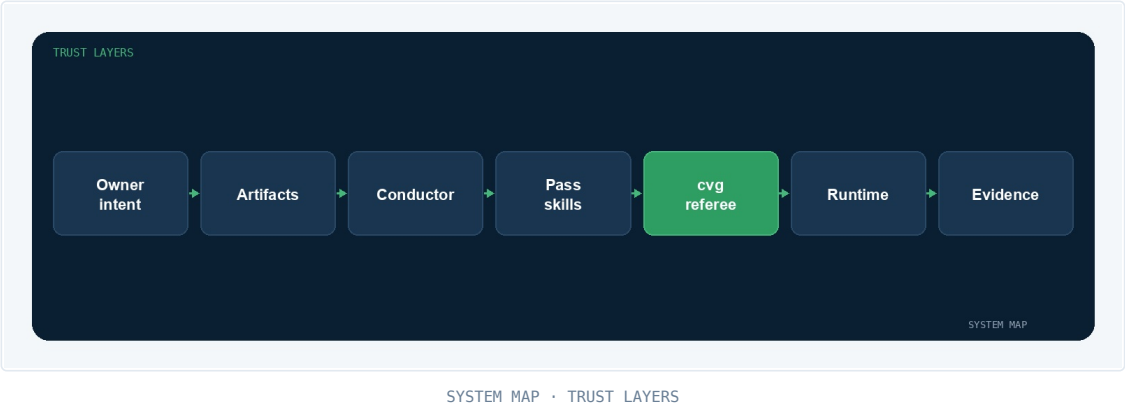
## Machine authority

Scripts lint, hash, check paths, run evals, emit stable tokens. Agents act only inside the artifact and capability boundary of their seat.

FOUNDATIONS · 04

# Six trust layers and one conductor

No single layer can silently redefine the others.

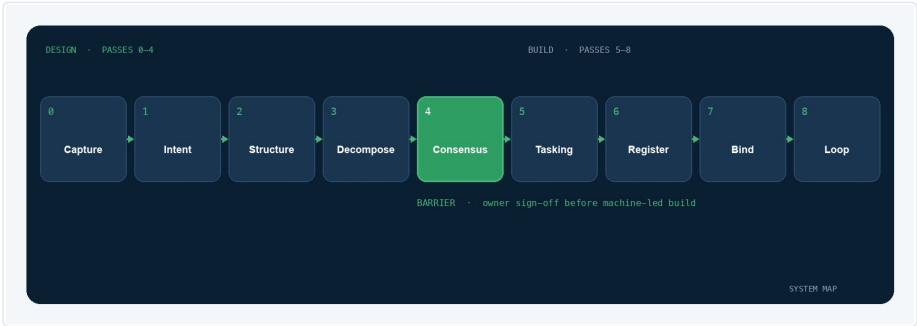


| LAYER         | OWNS                                    | MUST NOT PRETEND TO OWN            |
|---------------|-----------------------------------------|------------------------------------|
| Owner         | Priorities, trade-offs, accepted risk   | Mechanical validation              |
| Artifacts     | BRD, spec, ADRs, plans, Task-Specs      | Runtime enforcement by declaration |
| Conductor     | Lane order, evidence board, prompts     | Passing a gate or choosing a lane  |
| Skills        | Pass workflow, transformations, gates   | Fleet scheduling                   |
| CLI / referee | Discovery, routing, stable tokens       | Rewriting proven pass logic        |
| Runtime       | Tools, paths, attempts, brakes          | Changing signed budget authority   |
| Evidence      | Evals, hashes, logs, receipts, verdicts | Replacing owner semantic review    |

Layer failure rule. Artifacts declare policy; only a runtime enforces it. A runtime executes; only signed artifacts authorize. Evals prove declared behavior; a different-family holdout may still expose Goodharting. Receipts attest—they cannot rewrite the input contract.

# Two phases, one barrier, nine passes

*Human-led design first; machine-led build only after Consensus closes.*



NINE-PASS PIPELINE · BARRIER AT CONSENSUS

**Design · 0–4**  
Capture, Intent, Structure, Decompose, Consensus. Humans own meaning.

**Barrier · 4**  
Different-family refutation, complete objection log, explicit owner approval.

**Build · 5–8**  
Tasking, Register, Bind, Loop inside signed, hash-bound contracts.

Twelve skills total. **Nine** form the spine: idea-to-brd, brd-docs-to-tech-req, tech-req-to-adrs, reqs-to-swimlane-plans, sketch-plans-adversarial-review, task-spec, task-specs-to-issues, task-to-runtime-contract, task-loop. **Three** utilities: evidence-to-next-pass (conductor), pass-to-lesson, skill-creator.

| BOUNDARY | AUTHORITY AT EXIT                           | PROOF FORWARD                        |
|----------|---------------------------------------------|--------------------------------------|
| 0–1      | Owner approves problem, outcome, scope      | BRD + falsifiable tech spec          |
| 2–3      | Engineering grounds irreversible choices    | ADRs, vocabulary, swimlanes, legs    |
| 4        | Owner accepts plans + residual risk         | Fresh objection log + sign-off       |
| 5–7      | Tier-1 task defines authority; Bind derives | Spec, hashes, profile, guards, brief |
| 8        | Only terminal effects supported by evidence | Eval, verdict, commit/PR, receipt    |

## FOUNDATIONS · 06

# Routes choose ceremony; never waive proof

*`cvg lane` classifies risk. It does not execute, choose a task, or relax a gate.*

`cvg lane <intent>` is offline and deterministic. It evaluates kind of change, expected file surface, and risk markers, then emits one token: `LANE=FAST|NORMAL|FULL|USAGE_ERROR`. Pass the route to `cvg next --lane ...`. The router answers which passes apply; the conductor answers which of those comes next.

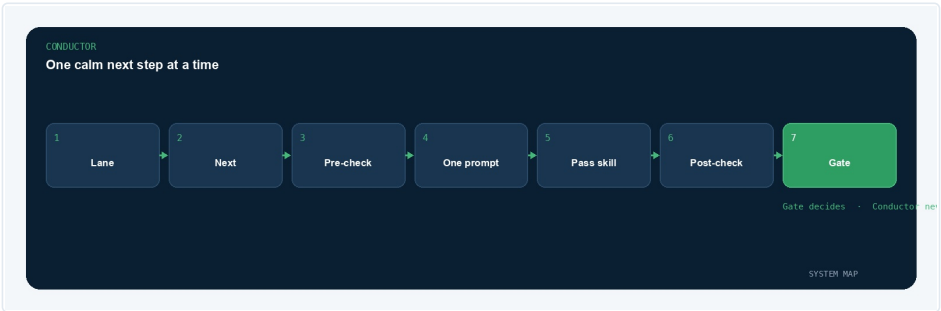
| LANE   | PASS SHAPE        | WHEN                                          |
|--------|-------------------|-----------------------------------------------|
| FAST   | 5 > 7 > 8         | Small reversible change in an existing seam   |
| NORMAL | 1 > 2 > 5 > 7 > 8 | Intent or grounding needs clarification       |
| FULL   | 0 > 8             | New system/seam, or risk/file-count escalates |

Risk overrides convenience. Auth, money, migrations, secrets, public APIs, irreversible ops, and new architectural seams cannot ride FAST. Escalating ceremony is always allowed; downgrading needs settled upstream evidence.

```
cvg lane "correct a typo in help string" --files 1      # LANE=FAST
cvg lane "migrate stored credentials" --files 6         # LANE=NORMAL
cvg lane "introduce new authentication seam" --files 6  # LANE=FULL
cvg next --lane FAST                                   # NEXT_PASS=N | DONE
```

# The conductor: one calm next step

*Read-only choreography. Never grants a PASS.*



CONDUCTOR SEQUENCE

- 01 Select the route**  
Human accepts FULL, NORMAL, or FAST from cvg lane. Conductor never downgrades.
- 02 Read the evidence board**  
cvg next --lane checks artifact homes; returns NEXT\_PASS=N or DONE.
- 03 Open the door**  
pre N refuses when earlier lane artifacts are absent.
- 04 Steer with one prompt**  
Agent reads cvg/brain/\_prompts/pass-N-\*.md—not improvised memory.
- 05 Leave the artifact home**  
Pass skill writes under typed cvg/docs-\*, sketch, tasks, execution, receipts.
- 06 Check presence, then truth**  
post N confirms landing; the pass’s own cvg gate gives the verdict.
- 07 Advance or repair**  
Green returns to cvg next. Failed gate stays at owning altitude.

THE INVARIANT. Evidence presence is not a verdict. A BRD file can move the board; only cvg capture can say whether that BRD passes. The conductor can refuse forward motion; it can never grant it.

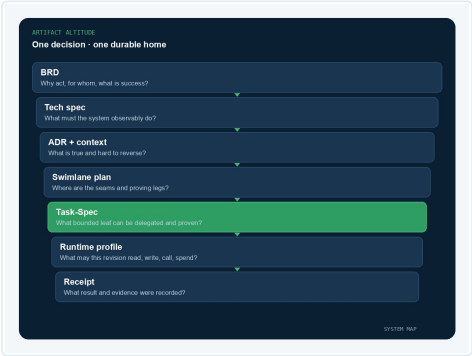
| SIGNAL            | PROVES                                    | DOES NOT PROVE                      |
|-------------------|-------------------------------------------|-------------------------------------|
| NEXT_PASS=4       | Earlier artifacts present; Consensus next | Those artifacts passed their gates  |
| CONDUCTOR_PRE=OK  | Prior lane evidence exists                | Pass N is semantically authorized   |
| CONDUCTOR_POST=OK | Pass N left its expected artifact         | The artifact is correct or accepted |



FOUNDATIONS · 08-09

# Artifact altitude and workspace

*One decision, one durable home. Resolve relative to the project workspace.*



| ARTIFACT      | PRIMARY QUESTION             | WRONG CONTENT HERE         |
|---------------|------------------------------|----------------------------|
| BRD           | Why act, for whom, success?  | Architecture, file paths   |
| Tech spec     | What must observably happen? | Unfalsifiable aspiration   |
| ADR + context | What is hard to reverse?     | Build plan as decision     |
| Swimlane plan | Where are seams and legs?    | Atomic tickets or code     |
| Task-Spec     | What leaf can be proven?     | New product decisions      |
| Profile       | What may this revision do?   | Replacement intent         |
| Receipt       | What was recorded?           | Predictions outside schema |

## FOUNDATIONS · 10

# A pass is a transformation contract

*Inputs, method, outputs, gate, honest failure route.*

**01 Inputs**

Canonical upstream artifacts and allowed live-system evidence.

**02 Transformation**

Skill workflow stating questions, altitude, and non-goals.

**03 Outputs**

Durable files that become the next pass's bounded input.

**04 Gate**

Deterministic checks with a greppable token and exit-code contract.

**05 Failure route**

Named stop or return to the pass that owns the missing decision.

The conductor wraps this contract but does not alter it. `CONDUCTOR_POST=OK` and a failing pass token are not contradictory: one proves presence, the other evaluates quality. `CHECK_ADR=OK` is structural; it does not mean the owner chose the right architecture. `CHECK_CONSENSUS=OK` proves the log is complete and fresh—the owner must still accept risk.

| GATE CLASS            | TYPICAL SUCCESS                       | TYPICAL RECOVERY                            |
|-----------------------|---------------------------------------|---------------------------------------------|
| Structure / lint      | Required sections at correct altitude | Repair owning artifact; rerun               |
| Freshness / integrity | Live bytes match hashes or HMAC       | Review, then renew stamp                    |
| Parity / graph        | Projection matches canonical specs    | Rebuild projection; never edit specs to fit |
| Static runtime        | Hashes, topology, guards coherent     | Repair or re-bind profile                   |
| Outcome               | Eval, scope, holdout satisfied        | Iterate or named non-success                |

Default unattended path: fail closed, explain precisely. Missing input, stale hashes, malformed evals must not become hopeful passes. Terminal tokens must name the outcome class so automation never parses narrative prose.

DESIGN · PASS 0 · IDEA-TO-BRD

# Capture — does this idea deserve a build?

*Optional when a usable BRD exists; essential when input is only a thought or pain.*

STEER pass-0-capture.md

CLOSE cvg capture

Pass 0 turns an unstructured idea into an **owner-voice business requirements document**—or a durable **no-go**. It asks frontier questions: every question whose prerequisites are settled appears in the same round, numbered, with a recommended default. Facts are looked up where possible; only decisions return to the owner. The BRD stays at business altitude. It records the affected actor, quantified pain, cost of inaction, target outcomes, KPIs in the owner's terms, explicit scope in and out, constraints, assumptions, and open questions with owners. It does **not** select frameworks, schemas, or file paths. The do-nothing test is first-class: if tolerable inaction beats the expected value of change, write a no-go—not a forced project.

| FIELD         | CONTRACT                                                             |
|---------------|----------------------------------------------------------------------|
| INPUT         | Idea, request, pain report, or incomplete brief                      |
| OUTPUT        | Owner-voice BRD under docs/brd/, or no-go under docs/no-go/          |
| GATE          | CHECK_BRD = PASS   FAIL   DRAFT_OK   DRAFT_INCOMPLETE   NOGO_OK      |
| EXIT EVIDENCE | Provenance-tagged pain, owner KPIs, non-empty scope, owned questions |

The checker strips fenced examples before looking for authorization, validates real ISO dates, rejects empty scope bullets, and requires every numbered pain measure to carry (measured), (estimated), or (guessed). Every guessed number must create an owned verification question. Owner voice remains human judgment; the gate warns rather than pretending grep can decide it.

```
cvg capture --draft cvg/docs/brd/observability.md
# CHECK_BRD=DRAFT_OK | DRAFT_INCOMPLETE
cvg capture cvg/docs/brd/observability.md
# CHECK_BRD=PASS | FAIL
cvg capture --no-go cvg/docs/no-go/observability.md
# CHECK_BRD=NOGO_OK | NOGO_INVALID
```

# Intent — falsifiable requirements

*Observable behavior, acceptance, and the decisions that materially change the build.*

STEER

pass-1-intent.md

CLOSE

cvg intent

Pass 1 reads the BRD as a **senior engineer**, not a transcriptionist. It identifies the real problem beneath the requested solution, asks the few questions whose answers change the build, and crystallizes a technical requirements specification whose claims can be proven or refuted. Every requirement traces to a business outcome or constraint. Acceptance criteria use observable behavior and measurable thresholds. Scope is explicit. Non-functional requirements state measurement methods—not adjectives like “fast” or “scalable.” Open decisions have owners; assumptions are marked as assumptions. The document is solution-aware only where a decision is genuinely required—it must not smuggle a full implementation plan.

**Good requirement**  
Names actor, trigger, observable response, boundary conditions, and a verification method tied to a BRD KPI.

**Weak requirement**  
Names a preferred tool, promises “robustness,” or declares completion without a falsifiable observation.

| FIELD           | CONTRACT                                                     |
|-----------------|--------------------------------------------------------------|
| INPUT           | Usable BRD plus supporting source material                   |
| OUTPUT          | Verifiable technical requirements specification              |
| GATE            | CHECK_Tech_SPEC = PASS   FAIL   DRAFT_*   USAGE_ERROR        |
| RETURN UPSTREAM | Outcome, owner, KPI, or material product decision unresolved |

**Gap register**  
Each unknown is a typed record with severity, owner, resolution. Blank or “pending” resolutions fail closed.

**Prior art**  
Research comparable solutions at the problem level. Do not import a vendor stack merely because another system used it.

```
cvg intent --draft cvg/docs/tech-spec/observability.md
cvg intent cvg/docs/tech-spec/observability.md
# CHECK_Tech_SPEC=PASS | FAIL | USAGE_ERROR
```

DESIGN · PASS 2 · TECH-REQ-T0-ADRS

# Structure — ground against the real system

*ADRs preserve hard-to-reverse decisions; CONTEXT.md preserves shared vocabulary.*

STEER pass-2-structure.md

CLOSE cvg structure

Pass 2 reconciles the specification with live terrain. In brownfield work it opens schemas, services, jobs, data flows, deployment boundaries, and existing contracts end to end. In greenfield work it names the assumptions the future system will stand on. It records only decisions that meet **three tests**: difficult to reverse, consequential for downstream work, and plausibly decidable another way. Each numbered ADR states context, decision, alternatives, consequences, and status. The Decision section says what is true; it must not become a disguised implementation checklist. Domain terms enter **CONTEXT.md** as they stabilize so later passes do not invent competing names.

| FIELD           | CONTRACT                                                      |
|-----------------|---------------------------------------------------------------|
| INPUT           | Technical requirements plus the live system                   |
| OUTPUT          | cvg/docs/adrs/* and CONTEXT.md                                |
| GATE            | CHECK_ADR = OK   FAIL   EMPTY   USAGE_ERROR                   |
| STRONG EVIDENCE | Paths, schemas, interfaces, observed jobs, authoritative docs |

Ground truth before plans. If a source is stale or the live system contradicts the spec, reconcile now. Pass 3 may decompose accepted reality; it may not choose architecture that Pass 2 declined to record. EMPTY means no decision artifact was discovered—it cannot authorize decomposition.

## Live-system owner

Engineering opens the executable path—not diagrams or README claims that may have drifted.

## Recovery

If an ADR says “build X,” lift the plan out. If it contradicts reality, repair the ADR. If reversible and local, remove it from the set.

```
cvg structure --final --dir cvg/docs/adrs
# CHECK_ADR=OK | FAIL | EMPTY | USAGE_ERROR
```

# Decompose — seams, swimlanes, legs

*A swimlane is a genuine system seam; a leg is an independently provable stretch.*

STEER pass-3-decompose.md

CLOSE cvg decompose

Pass 3 turns grounded requirements into a plan tree **without falling to task or code altitude**. It finds joints already present—feature boundaries, components, data ownership, protocols, operational responsibility—and creates one focus-oriented plan per genuine seam under `cvg/sketch/swimlane-*/`. The chain is **seam > swimlane > leg > Task-Spec**. A lane owns one coherent responsibility. Each leg names a bounded change, dependencies, inherited ADRs, and one proving test. A steel thread crosses the minimum set of lanes needed to demonstrate end-to-end value early. The right number of lanes is the number of real seams.

### Plan altitude

Responsibilities, boundaries, ordering, contracts, risks, proving tests, integration points.

### Too low

Atomic tickets, SQL, function signatures, line edits, worker prompts—those belong downstream.

| FIELD           | CONTRACT                                               |
|-----------------|--------------------------------------------------------|
| INPUT           | Accepted requirements, ADRs, shared vocabulary         |
| OUTPUT          | Swimlane plan tree with named legs and a steel thread  |
| GATE            | CHECK_PLAN = OK   FAIL   EMPTY   USAGE_ERROR           |
| RETURN UPSTREAM | A seam depends on an unrecorded architectural decision |

Each swimlane directory contains a lean plan index and contiguous leg files. The stable leg key excludes the swappable technology label. Legs carry Given/When/Then proof seeds, independence, interfaces, appetite, and the 1:N Task-Specs they will yield—never executable eval code. Pass 3 creates the surface Pass 4 attacks; blended seams make adversarial review meaningless.

```
cvg decompose --dir cvg/sketch
# CHECK_PLAN=OK | FAIL | EMPTY | USAGE_ERROR
```

DESIGN · PASS 4 · SKETCH-PLANS-ADVERSARIAL-REVIEW

# Consensus — try to refute

*Agreement is earned by surviving attack—not by asking the author to self-review.*

STEER pass-4-consensus.md

CLOSE cvg review --check

Pass 4 binds a **different-family engine as a skeptic**. The adversary begins from “refuted,” attacks plans one at a time, and looks for broken assumptions, missing dependencies, unsafe boundaries, unverifiable legs, contradictory ADRs, Goodhartable tests, and failure modes the author normalized away. `cvg review --adversary <engine>` dispatches through a read-only adapter. Multiple adversaries merge into one referee-stamped log. The referee holds **no model credentials**; each engine CLI authenticates itself. Input hashes and adversary provenance are recorded so the log cannot be reused after plans change. Multi-adversary mode fails closed if no cross-family engine actually ran.

| RESULT             | MEANING                                                  |
|--------------------|----------------------------------------------------------|
| REVIEW=OK          | Cross-family attack completed; stamped result produced   |
| REVIEW=SKIP        | No eligible engine ran—this is not consensus             |
| REVIEW=TIMEOUT     | Attack did not finish inside the watchdog                |
| REVIEW=ERROR       | Dispatch or merge could not establish trustworthy output |
| REVIEW=USAGE_ERROR | Invocation contract invalid                              |

```
cvg doctor                # DOCTOR=OK | FAIL
cvg review --adversary codex,kimi # REVIEW=OK | ERROR | SKIP | ...
cvg review --check         # CHECK_CONSENSUS=OK | FAIL | EMPTY
```

Closure happens only after every objection is resolved and live plans still match stamped provenance. Absence of objections is not evidence that none exist if dispatch skipped or timed out.

DESIGN · BARRIER AFTER PASS 4

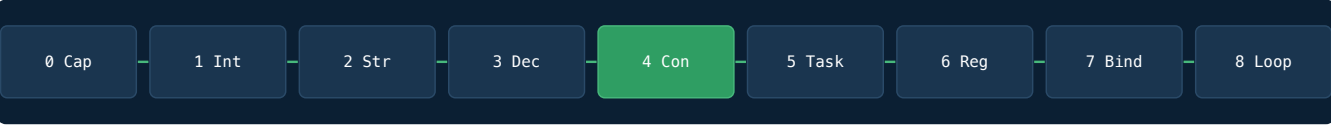
# Close every objection, then ask the owner

*The machine proves the log is complete and fresh; only the owner accepts residual risk.*  
Every Pass 4 objection has one of two legitimate dispositions: **FIXED** (plan changed; response points to the repair) or **ACCEPTED** (risk remains intentionally, with named owner and rationale). “Noted,” “later,” silent deletion, or an unowned exception is still open.

The barrier has two locks. (1) Deterministic gate: no unresolved, malformed, or stale objection survives. (2) Owner explicitly signs off hardened plans and any accepted residual risk. Crossing flips the operating mode: upstream remains human-led design; downstream becomes machine-led build inside those constraints.

| OBJECTION FIELD   | REQUIRED MEANING                                              |
|-------------------|---------------------------------------------------------------|
| Identity + source | Stable number, plan/leg reference, adversary provenance       |
| Claim + evidence  | Concrete failure or contradiction + supporting artifact       |
| Disposition       | FIX with changed location, or ACCEPT with owner and reason    |
| Freshness         | Input hashes recomputed against every current plan in the set |

Changing a plan during resolution is expected; closing the barrier on the old hash is not. The final check must observe the repaired bytes. An owner signature recorded before those checks cannot be carried forward as approval of the final plan set. If Pass 5 exposes a new seam or needs a product decision, the barrier was incomplete—return upstream, renew attack, sign again. Do not patch semantic gaps inside a task.





# Tasking — make the definition of done travel

*Atomic, vendor-neutral units: behavior, scope, budgets, and runnable proof together.*

STEER   pass-5-tasking.md   CLOSE   cvg tasks gate --stamp

Pass 5 turns accepted plan legs into a dependency DAG of leaf tasks under `cvg/tasks/`. A runnable leaf declares goal, behavior, scope, dependencies, execution choices, budgets, evals, and an Exit Check. The implementer receives the **same definition of done** the gates later use—completion criteria cannot be reconstructed from a chat transcript.

| EFFORT   | SHAPE                      | WRITE-SURFACE GUIDELINE         |
|----------|----------------------------|---------------------------------|
| XS       | Runnable leaf              | 1 path                          |
| S        | Runnable leaf              | 2 paths                         |
| M        | Runnable leaf              | 3 paths                         |
| L        | Long-horizon runnable leaf | 5 paths                         |
| XL / XXL | Decomposition node         | Children required; never a leaf |

Three authoring profiles: `lite`, `standard` (default), `full`. Path counts warn above threshold in 0.1.0; L requires a configured long-horizon backend. PRE seals delegation authority; POST proves resulting work. They are duals—not aliases. A broken eval is an upstream defect: report and repair, never weaken signed goalposts inside execution.

| FIELD     | CONTRACT                                                                   |
|-----------|----------------------------------------------------------------------------|
| INPUT     | Accepted swimlane legs and inherited decisions                             |
| OUTPUT    | Task-Spec DAG with runnable evals and explicit dependencies                |
| PRE GATE  | <code>safe-to-delegate.sh --stamp</code> ; Tier 1 for unattended execution |
| POST GATE | <code>accept-task.sh --stamp</code> ; work, Git scope, integrity           |

```
cvg tasks new export-format S
cvg tasks validate --no-state cvg/tasks/T-...-export-format.md
cvg tasks gate --stamp cvg/tasks/T-...-export-format.md
# TIER=1 (required for unattended execution)
```

# Six zones of a Task-Spec

*YAML frontmatter for machine control; six human-readable zones for work meaning.*

| ZONE                         | PURPOSE                                                        |
|------------------------------|----------------------------------------------------------------|
| 1 · Intent                   | Goal, rationale, inherited decisions, explicit non-goals       |
| 2 · Behavior                 | Numbered observable behaviors and boundary cases               |
| 3 · Contract                 | Success criteria, runnable evals, validation card, Exit Check  |
| 4 · Guardrails               | Allowed paths, forbidden actions, dependency and safety limits |
| 5 · Operations               | Open questions, honest unknowns, blocker conditions            |
| 6 · Rollback + observability | Full-profile recovery, signals, failure visibility             |

Frontmatter carries identity and control: task ID, status, profile, effort, dependencies, write/create paths, backend and agent choices, budgets, sign-off envelope, acceptance state, optional tracker reference. The PRE HMAC covers body bytes and authority-bearing frontmatter. Changing any of them creates a new delegation decision. The `profile` field itself is not HMAC-covered—changing it after Bind changes the whole-file hash and invalidates the epoch.

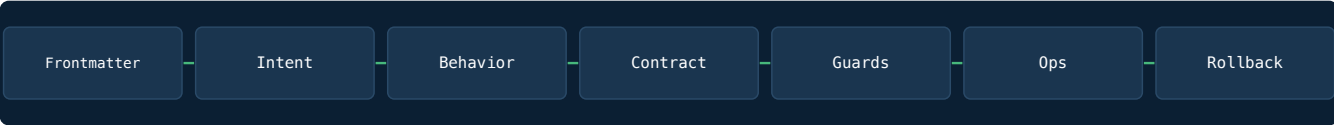
### Profiles

lite for small reversible leaves; standard for full behavior/eval traceability; full for material blast radius recovery detail.

### Lifecycle

ready > in-progress > blocked|done|parked. done requires stamped accepted: true plus matching result: pass receipt.

Runnable evals must fail before the promised behavior, inspect the public contract, and emit diagnostic evidence. Destructive or network-dependent setup needs explicit fixtures or blocks delegation. The Exit Check composes the terminal claim—it is not another prose criterion. `cvg tasks dod <spec>` renders definition of done and returns DOD=COMPLETE|GAPS.



BUILD · PASS 6 · TASK-SPECS-TO-ISSUES · OPTIONAL

# Register — project the backlog, never replace it

*An opt-in operational shadow of canonical Task-Spec files.*

STEER pass-6-register.md

CLOSE cvg register --check

Pass 6 maps one signed-off Task-Spec to one tracker issue and every `depends_on` edge to one `blocked-by` relationship.

Supports GitHub, Linear, Jira, and a fake adapter for offline tests. Registration is **idempotent**: the issue-side marker is the identity key, so reruns update rather than duplicate.

The projection may stamp a `tracker_ref` receipt back into the spec while the issue links to a branch-pinned source URL. The referee does not hold tracker credentials; the selected adapter authenticates itself. A six-verb adapter boundary keeps tracker-specific APIs outside the canonical method. Pass 6 is optional because a repo-local ready frontier is sufficient for single-task operation.

| COMMAND               | CONTRACT                                                                          |
|-----------------------|-----------------------------------------------------------------------------------|
| cvg register          | Project sealed specs; dry-run emits DRY_RUN=OK; normal run REGISTER=OK FAIL EMPTY |
| cvg register --check  | Read-only 1:1 parity, edges, cycles, leak check; CHECK_REGISTER                   |
| cvg setup tracker ... | Validate and record adapter choices—never credentials                             |

Register filters by `signed_off`: true and `registerable`: true only—it does not validate HMAC/tier. Hand-stamped specs can be projected unless validation ran upstream. A mismatch is recovered by replaying projection from canonical files—not by editing `depends_on` to match a hand-edited board.

```
cvg register --tracker linear --dry-run # DRY_RUN=OK
cvg register --tracker linear          # REGISTER=OK | FAIL | EMPTY
cvg register --check                   # CHECK_REGISTER=OK | FAIL
```

BUILD · PASS 7 · TASK-TO-RUNTIME-CONTRACT

# Bind — one signed task to an honest runtime

*7A emits the enforceable contract; 7B emits the minimal brief the worker reads.*

STEER pass-7-bind.md

CLOSE cvg bind --check

Pass 7 resolves and verifies one runnable Task-Spec, binds that exact revision to the smallest evidence slice needed, chooses an initial task-local topology, generates portable path guards and runtime adapter manifests, and gates the result. Profile lives under `cvg/execution/<task-id>/`.

**7A — execution-profile.yaml.** Names the epoch, spec hash, evidence hashes, topology, writable and creatable paths, required capabilities, external-write policy, a hash-bound pointer to Task-Spec budgets, and the mechanism by which an adapter claims to prevent, detect, or cannot honor a constraint. Source-hash-bound control-plane state—not independently signed authority.

**7B — AGENTS.task.md.** Compact model context: IDs, Goal, exact allowed and forbidden paths, the actual Exit Check, network and external-write boundaries, assurance, closure instructions, and a pointer to the project router. Not identifier-only; does not synthesize broad doctrine. The worker must read canonical, hash-bound sources from disk.

| FIELD         | CONTRACT                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------|
| DEFAULT INPUT | Tier-1 signed leaf (--supervised can bind Tier 2 but does not prove a supervisor at Loop launch) |
| OUTPUT        | Execution profile, guards, adapter manifests, task brief                                         |
| STATIC GATE   | CHECK_RUNTIME_CONTRACT=PASS FAIL for profile, hashes, topology, manifests, guards                |
| HOST PROBE    | cvg doctor runtime-contract — separate; not auto-fed into Bind or Loop                           |

```
cvg doctor runtime-contract --runtime generic
# DOCTOR_RUNTIME_CONTRACT=OK | DEGRADED | FAIL
cvg bind --task cvg/tasks/T-...-export-format.md
# CHECK_RUNTIME_CONTRACT=PASS | FAIL
cvg bind --check --task cvg/tasks/T-...-export-format.md
```

## BUILD · CAPABILITY ENVELOPE

# Authority as an epoch

*Runtime authority is `<task-id>@<spec-sha12>`. Re-bind after any spec byte changes.*

Capabilities are scoped to that epoch and the Task-Spec's own paths. The profile declares `settle`, `block`, `budget_exhausted`, and `epoch_change` as intended closure events. Release 0.1.0 does **not** persist a revoked state; a terminal token does not invalidate the profile. Operators must close authority and re-bind before reuse.

| ASSURANCE    | MEANING                                     | EXAMPLE                             |
|--------------|---------------------------------------------|-------------------------------------|
| Prevent      | Stops the violation before it occurs        | OS sandbox, pre-tool hook, worktree |
| Detect       | May allow; postflight can refuse settlement | Portable diff/path guard            |
| Cannot honor | No truthful mechanism for the control       | Unavailable isolation or network    |

In default bound mode, write scope is least privilege. Settlement stages only paths granted by `fs.write` —never broad `git add -A`. Network egress is denied unless the task justifies it. Headline assurance is the **weakest** required capability, so one unenforced grant cannot hide behind several prevented controls. Silence is not a waiver. “Sandboxed” is not evidence unless mapped to a concrete host control.

An unenforced waiver is visible only in the unsigned derived profile: no owner, timestamp, or signature in 0.1.0, and not copied into the receipt. Record owner approval separately. Credentials stay at the harness/host boundary.

```

cvg bind --task ... --runtime codex --require net.egress
# static PASS only when adapter declares the mechanism
cvg bind --task ... --runtime generic --require net.egress --accept-unenforced
# explicit WAIVED record; assurance becomes unenforced

```

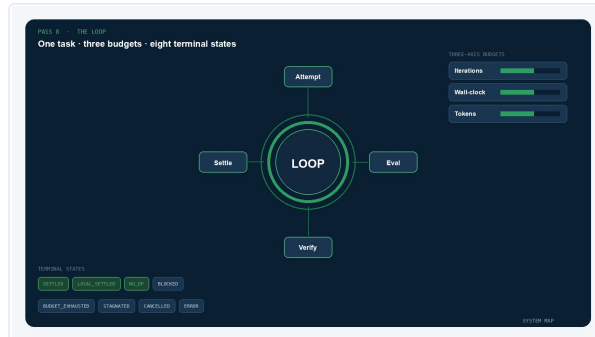
## BUILD · PASS 8 · TASK-LOOP

# The Loop — one assigned task, driven to evidence

*Attempt > verify > repeat under three-axis budgets. Only three states exit zero.*

STEER pass-8-loop.md

CLOSE cvg loop



LOOP CONTROL ROOM · BUDGETS AND TERMINAL STATES

`cvg loop` runs under budgets for **iterations**, **wall-clock**, and **tokens**, with a stagnation detector and eight named terminal states. Fresh context each attempt. Command-line flags may tighten resolved limits but cannot raise them. Pre-attempt brakes recheck Bind freshness, path policy, and host readiness before each write cycle. Settlement stages only paths granted by `fs.write`. A receipt-write failure does not by itself revoke a `SETTLED` token in 0.1.0—treat that combination as incomplete evidence and refuse lifecycle close without a parseable matching receipt. No dry-run branch commits, pushes, opens a PR, or writes a receipt.

```
cvg loop --task cvg/tasks/T-...-export-format.md --agent codex
# TASK_LOOP=SETTLED | LOCAL_SETTLED | NO_OP | BLOCKED | BUDGET_EXHAUSTED | ...
```

## BUILD · EIGHT TERMINAL STATES

# Failure must be legible

*Only `SETTLED`, `LOCAL_SETTLED`, and `NO_OP` exit zero.*

| STATE            | EXIT | MEANING                                                                      |
|------------------|------|------------------------------------------------------------------------------|
| SETTLED          | 0    | Evals green, integrity ok, optional Tier-2 upheld; receipt; publish complete |
| LOCAL_SETTLED    | 0    | Work proven locally; outward publish deferred by policy                      |
| NO_OP            | 0    | Nothing to do; already satisfied against current contract                    |
| BLOCKED          | !=0  | Dependency or external condition prevents progress                           |
| BUDGET_EXHAUSTED | !=0  | Iteration, time, or token budget hit without settlement                      |
| STAGNATED        | !=0  | Repeated attempts produced no meaningful progress                            |
| CANCELLED        | !=0  | Operator or policy cancelled mid-flight                                      |
| ERROR            | !=0  | Runtime, integrity, or verification failed safely                            |

Always interpret mode together with the token. If publication or another outward leg fails, recover from the scoped commit and recorded error rather than rerunning implementation blindly. The separate `transition ... done` guard refuses completion unless a parseable matching `result: pass` receipt actually exists.

Loop has brakes on three axes plus stagnation. An exhausted budget, blocked dependency, or runtime error must remain distinct non-zero outcomes so automation never collapses them into a hopeful success.

BUILD · TIER 2 HOLDOUT

# Adversarial second reader

*Holdout omitted from the 7B brief; still readable in the canonical spec.*

`cvg verify --task <spec> --judge <engine>` asks a model to grade the diff against intent and a Holdout omitted from `AGENTS.task.md`. This is **review diversity**, not enforced train/test secrecy—Loop still tells the worker to open the canonical Task-Spec where Holdout is readable. Auto-selection compares judge family with a recognized `execution_backend`.

| VERDICT                  | RUNTIME MEANING                                     |
|--------------------------|-----------------------------------------------------|
| CHECK_VERIFY=UPHELD      | Judge found work consistent with intent and holdout |
| CHECK_VERIFY=REFUTED     | Concrete violation found; settlement refused        |
| CHECK_VERIFY=UNAVAILABLE | No installed judge established a verdict            |
| CHECK_VERIFY=ERROR       | Verification process failed safely                  |

Release 0.1.0 demonstrated both outcomes: one cross-family holdout **upheld** work; another **refuted** a green eval whose error path reported false health. The kernel refused settlement and wrote a handoff—proof that independent review can change the terminal state. Risk policy decides whether unavailable verification is tolerable. Loop has no passthrough for `--accept-unverified`.

Missing verifier gap. If the verifier file itself is missing, Loop currently prints that Tier 2 is skipped and continues. Prelaunch policy must verify the file exists when second review is required. Inspect recorded independence: independent | same-family | none.

```
cvg verify --task cvg/tasks/T---export-format.md --judge kimi
# CHECK_VERIFY=UPHELD | REFUTED | UNAVAILABLE | ERROR
```



# Sign intent. Bind authority. Verify work.

*cvg holds zero model credentials and never writes product code.*

## 01 Sign intent

HMAC seals Task-Spec body and authority-bearing frontmatter before unattended work.

## 02 Bind authority

Profile derives least-privilege paths, capabilities, and budget pointers from that sealed revision.

## 03 Verify work

Evals prove declared behavior; optional different-family holdout attacks residual Goodharting.

## 04 Preserve evidence

Receipts, logs, terminal tokens, and hashes form an inspectable chain—not a narrative claim.

Engine CLIs and tracker adapters authenticate themselves. The referee only frames, dispatches headlessly, and gates. Because the referee is plain bash + stdlib Python, the same install serves Claude Code, Codex, Kimi, and Grok Build. Vendors enter through flags and adapter manifests; they are not baked into the method.

### What persists

Canonical artifacts under `cvg/docs`, sketch, tasks; signed specs; profiles; loop logs; receipts.

### What is derived

Indexes, tracker projections, agent briefs, path guards, adapter manifests—rebuildable from sources.

Setup keeps credentials out of project state. `.cvg/config` records backend, team, and projection choices—never API keys. Skills install into each harness's native discovery directory.

## OPERATING SYSTEM · REFEREE SURFACE

# The CLI is referee, router, and conductor

*Plain bash + stdlib Python. Zero runtime dependencies. Harness-agnostic by construction.*

| DOMAIN  | COMMANDS                                          | STABLE TOKENS                                     |
|---------|---------------------------------------------------|---------------------------------------------------|
| Design  | capture · intent · structure · decompose · review | CHECK_BRD · TECH_SPEC · ADR · PLAN · CONSENSUS    |
| Route   | lane · next · init                                | LANE=... · NEXT_PASS=N · DONE                     |
| Tasks   | tasks new validate gate dod                       | TIER=1 · DOD=COMPLETE GAPS                        |
| Project | register · setup tracker                          | REGISTER=OK · CHECK_REGISTER                      |
| Runtime | bind · doctor · loop · verify                     | CHECK_RUNTIME_CONTRACT · TASK_LOOP · CHECK_VERIFY |
| Meta    | help · agent-context · version                    | JSON envelope · command manifest                  |

Command output and exit status are a pair. A stable token makes semantic state greppable; the exit code controls orchestration. Draft, empty, usage, blocked, and exhausted outcomes remain distinct even when several are non-zero. Wrappers preserve byte output and publish the taxonomy through `cvg agent-context`.

```
cvg help
cvg agent-context      # machine-readable command + token taxonomy
cvg version
cvg init               # seed nine canonical pass prompts
```

# Commands for Passes 0–4

*Confirm against the installed 0.1.0 package before automating.*

| COMMAND                              | PASS   | PRIMARY TOKENS                     |
|--------------------------------------|--------|------------------------------------|
| cvg capture [--draft --no-go] <path> | 0      | CHECK_BRD=PASS FAIL DRAFT_* NOGO_* |
| cvg intent [--draft] <path>          | 1      | CHECK_TECH_SPEC=PASS FAIL DRAFT_*  |
| cvg structure --final --dir <adrs>   | 2      | CHECK_ADR=OK FAIL EMPTY            |
| cvg decompose --dir <sketch>         | 3      | CHECK_PLAN=OK FAIL EMPTY           |
| cvg doctor                           | 4 prep | DOCTOR=OK FAIL                     |
| cvg review --adversary <engines>     | 4      | REVIEW=OK SKIP TIMEOUT ERROR       |
| cvg review --check                   | 4 gate | CHECK_CONSENSUS=OK FAIL EMPTY      |

Design commands discover candidates under typed `cvg/docs/*` folders first, then legacy flat prefixes. Zero or multiple candidates are safe failures that report the search. Explicit paths always win. After any upstream edit, renew dependent review, signature, profile, or receipt claims before claiming progress.

# Commands for Passes 5–8

*PRE seals authority; POST and Loop prove work; verify attacks residual risk.*

| COMMAND                              | PASS    | PRIMARY TOKENS                           |
|--------------------------------------|---------|------------------------------------------|
| cvg tasks new validate gate dod      | 5       | TIER=1 · DOD=COMPLETE GAPS               |
| cvg register [--check --dry-run]     | 6       | REGISTER=OK · CHECK_REGISTER · DRY_RUN   |
| cvg bind [--check] --task <spec>     | 7       | CHECK_RUNTIME_CONTRACT=PASS FAIL         |
| cvg doctor runtime-contract          | 7 host  | DOCTOR_RUNTIME_CONTRACT=OK DEGRADED FAIL |
| cvg loop --task <spec> --agent ...   | 8       | TASK_LOOP=<terminal state>               |
| cvg verify --task <spec> --judge ... | 8 tier2 | CHECK_VERIFY=UPHELD REFUTED ...          |

Every task command must resolve the same backlog root. After task resolution, execution profiles, loop state, and eval working directories must all derive from the same workspace decision. A mismatch is a contract failure—it changes both inputs and possible side effects.

Tool root (CVG\_HOME) selects the installed package. Project root (CVG\_PROJECT\_ROOT) selects local .cvg state. The two roots are independent so one toolchain can serve many projects without confusing package assets with project state.

## PRACTICE · WALK A

# From operational pain to signed-off plan

*Design descent under FULL or NORMAL—stop at the barrier before any unattended build.*

**01 Capture the pain**

Owner-voice BRD with provenance-tagged measures, KPIs, explicit scope—or a durable no-go.

**02 Falsify intent**

Tech spec with observable acceptance criteria; open decisions owned; assumptions marked.

**03 Ground structure**

Minimal ADR set + CONTEXT.md against the live system. Record only hard-to-reverse choices.

**04 Cut seams**

Swimlane plan tree with legs, steel thread, proving tests—no code altitude.

**05 Attack and close**

Different-family review; resolve every objection; owner signs residual risk.

If any step surfaces a missing business decision, return to Capture or Intent. If the live system contradicts a planned architecture, return to Structure. If a seam is fake, return to Decompose. Do not let the Loop discover product meaning that Design left ambiguous.

```
cvg lane "improve observability of export pipeline" --files 12 # likely FULL/NORMAL
cvg next --lane FULL
# follow NEXT_PASS prompts; close each gate before advancing
cvg review --adversary codex,kimi && cvg review --check
# owner sign-off on hardened plans + accepted risk
```

## PRACTICE · WALK B

# From Task-Spec to independently checked settlement

*Build path after the barrier—or FAST when upstream intent is already settled.*

**01 Author and seal**

Leaf Task-Spec with runnable evals; PRE HMAC; Tier-1 for unattended work.

**02 Optional project**

Register to Linear/GitHub/Jira as a shadow—canonical file remains source of truth.

**03 Bind runtime**

Hash-bound profile + compact brief; doctor the host separately.

**04 Loop with brakes**

Fresh attempts under budgets; only SETTLED / LOCAL\_SETTLED / NO\_OP exit zero.

**05 Independent proof**

Optional Tier-2 holdout; write receipt; refuse lifecycle close without it.

```
cvg tasks new export-format S
cvg tasks gate --stamp cvg/tasks/T-...-export-format.md
cvg bind --task cvg/tasks/T-...-export-format.md
cvg doctor runtime-contract --runtime codex
cvg loop --task cvg/tasks/T-...-export-format.md --agent codex
cvg verify --task cvg/tasks/T-...-export-format.md --judge kimi
```

If Loop lands REFUTED, BUDGET\_EXHAUSTED, or STAGNATED, repair at the owning altitude. A broken eval returns to Tasking. A missing capability returns to Bind or host policy. A product ambiguity returns past the barrier—never invents a decision inside the attempt.

PRACTICE · EVIDENCE AND ROADMAP

# What o.i.o has actually demonstrated

*Current-state truth—not a roadmap presented as reality.*

| DEMONSTRATED                                                   | HONEST BOUNDARY                                        |
|----------------------------------------------------------------|--------------------------------------------------------|
| Eval-gated settlement; maximal stub attack lands all nine RED  | Receipt-write failure does not auto-revoke SETTLED     |
| Cross-family holdout UPHELD and REFUTED (caught fail-open bug) | Missing verifier file currently skips Tier-2           |
| HMAC sign-off seals body + authority frontmatter               | Profile field is not HMAC-covered                      |
| Three-axis loop budgets + eight terminal states                | Single-task Loop only—no fleet Manager                 |
| Portable skills across Claude, Codex, Kimi, Grok Build         | Engines authenticate themselves; cvg holds no keys     |
| Lane classifier + conductor next-pass sequencing               | Routing shipped; concurrent multi-task dispatch is not |

**Shipped now**

Nine pass spine, three utilities, cvg referee, Task-Spec v3, Bind profiles, Loop kernel, Tier-2 verify, agent-context envelope.

**Explicitly future**

Fleet Manager, enforced holdout secrecy, automatic revoked-authority state, deeper host sandbox integration.

Start with Task-Spec + evals on a single leaf. Add the conductor when multi-pass design needs a stable next step. Add Bind when unattended execution needs least privilege. Add Tier-2 when green evals are no longer enough to sleep well. Escalate ceremony when risk demands it—never waive proof to go faster.

## PRACTICE · VOCABULARY

# Words that keep the chain coherent

*Competing names for the same concept are a defect.*

| TERM      | MEANING IN CONVERGE                                           |
|-----------|---------------------------------------------------------------|
| Altitude  | Decision layer an artifact is allowed to occupy               |
| Barrier   | Pass 4 dual lock: machine-complete log + owner sign-off       |
| Conductor | Read-only sequencer that hands one next pass prompt           |
| Epoch     | Runtime authority window: task-id@spec-sha12                  |
| Gate      | Deterministic check with greppable token + exit-code contract |
| Holdout   | Criteria omitted from worker brief; used by Tier-2 judge      |
| Lane      | FAST / NORMAL / FULL route shape; never a proof waiver        |
| Receipt   | Recorded subset of execution evidence for a terminal outcome  |
| Referee   | cvg CLI—routes, scores, never plays                           |
| Task-Spec | Atomic signed unit: behavior + evals + budgets + authority    |
| Tier-1    | HMAC-sealed, PRE-gated authority for unattended execution     |
| Tier-2    | Different-family adversarial second reader on the result      |

Verify every contract against the installed 0.1.0 package. Executable source and tests outrank this handbook. Use this edition to understand; confirm against cvg help, skill SKILL.md files, and the live test suite before automation is committed.



# Compile intent into shipped software.

The dark factory model: autonomous agents do the building; machines do the checking. Done is proven by runnable evals, never claimed by the agent. One method, one referee CLI, twelve portable skills, zero runtime dependencies.

- > *Agreement descends*
- > *Authority narrows*
- > *Evidence accumulates*

PACKAGE 0.1.0 · EXPANDED EDITION · NAVY + GREEN  
[github.com/luanmorenommaciel/converge](https://github.com/luanmorenommaciel/converge)